Ref. Ares(2024)2361829 - 29/03/2024

Symbolic Logic Framework for Situational Awareness in Mixed Autonomy

Project funded by the European Innovation Council and SMEs Executive Agency

D4.1 An automated methodology for verifying knowledge awareness

Work Package: WP4

Authors: Arabinda Ghosh, Birgit Van Huijgevoort, Sadegh Soudjani

Project Title: Symbolic Logic Framework for Situational Awareness in Mixed Autonomy

Project Acronym: SymAware

Grant Agreement number: 101070802

Project Start Date: 1st October 2022

Duration: 36 months

Project Coordinator: Max-Planck-Gesellschaft Zur Forderung Der Wissenschaften EV (MPI-SWS)

Project Members:

Max-Planck-Gesellschaft Zur Forderung Der Wissenschaften EV (MPI-SWS)

Technische Universiteit Eindhoven (TUE)

Kungliga Tekniska Hoegskolan (KTH)

Uppsala Universitet (UU)

Stichting Koninklijk Nederlands Lucht - En Ruimtevaartcentrum (NLR)

Siemens Industry Software Netherlands BV (SISW)

Document Identifier: D4.1
Issue: 1.3
Issue Date: 18.03.2024
Prepared by: MPI-SWS
Approved: Yes
Dissemination Status: Confidential, only for members of the consortium
History Chart:

Issue	Date (DD/MM/YYYY)	Updated Sections	Cause of Change	Implemented by
1.0	06.02.2024	All sections	New Document	Arabinda Ghosh
1.1	08.03.2024	Chapter 2	Added content to the chapter	Birgit Van Huijgevoort
1.2	14.03.2024	Chapter 2	Added content to the chapter	Arabinda Ghosh
1.3	18.03.2024	All sections	revising and polishing	Sadegh Soudjani

All rights reserved

The document is proprietary of the SymAware consortium members listed on the front page of this document. The document is supplied on the express understanding that it is to be treated as confidential and may not be used or disclosed to others in whole or in part for any purpose except as expressly authorized in terms of Grant Agreement number 101070802. The SymAware consortium makes no warranty for the information contained in this document; neither does it assume any legal liability or responsibility for the accuracy, completeness or usefulness of this information. Company or product names mentioned in this document may be trademarks or registered trademarks of their respective companies.

Contents

List of Figures **3**

1 Introduction **4**
1.1 SymAware: High-level Description of the Project 4

2 Automated Methodology for Verification of Knowledge Awareness **5**
2.1 Knowledge Awareness for Single-agent System 5
2.1.1 Description Logic 5
2.1.2 Knowledge Awareness 9
2.1.3 Linear Temporal Logic Specification 10
2.1.4 Finite Transition System 10
2.1.5 Generalized Reactivity 10
2.1.6 Symbolic Controller Synthesis 11
2.2 Epistemic Logic for Multi-agent Systems 11
2.2.1 Literature 11
2.2.2 Research Directions 11

List of Figures

2.1 The proposed architecture for integrating knowledge awareness into a reactive controller synthesis. 6

1. Introduction

1.1 SymAware: High-level Description of the Project

SymAware addresses the fundamental need for a new conceptual framework for awareness in multi-agent systems (MASs) that is compatible with the internal models and specifications of robotic agents and that enables safe simultaneous operation of collaborating autonomous agents and humans. The goal of SymAware is to provide a comprehensive framework for situational awareness to support sustainable autonomy via agents that actively perceive risks and collaborate with other robots and humans to improve their awareness and understanding while fulfilling complex and dynamically changing tasks. The SymAware framework will use compositional logic, symbolic computations, formal reasoning, and uncertainty quantification to characterise and support situational awareness of MAS in its various dimensions, sustaining awareness by learning in social contexts, quantifying risks based on limited knowledge, and formulating risk-aware negotiation of task distributions. These objectives will be achieved in SymAware through

1. logical characterization of awareness using symbolic methods,
2. quantifying the symbolic reasoning for awareness with spatial and temporal ingredients for decision making,
3. risk awareness via quantified knowledge,
4. quantifying and communicating knowledge awareness,
5. demonstrating awareness engineering in aviation and automotive use cases, and
6. identifying requirements for ethical and trustworthy awareness in human-agent interaction.

The objectives of SymAware address the “Awareness Inside” Challenge of the European Innovation Council (EIC) by extending and formalizing human-based models of situational awareness and by providing a novel conceptual situational awareness framework for MASs that encompasses logical characterisation and integrative formal reasoning of interdependent awareness dimensions including knowledge, spatiotemporal, risk and social dimensions. This will support transitioning to safe mixed operation of autonomous agents and humans.

2. Automated Methodology for Verification of Knowledge Awareness

Knowledge awareness in MASs refers to the ability of an agent to comprehend, utilize, and adjust their actions based on pertinent knowledge and data. In MASs, the integration of computational components with knowledge empowers them to perceive their surroundings and make well-informed decisions for effective and secure operations. A prime example is evident in autonomous driving scenarios, where there is a continuous exchange of real-time information among sensors, cameras, and controllers. Any malfunction in this chain could lead to significant repercussions. Thus, it is imperative to systematically cultivate an understanding of the environment within the autonomous driving system, a practice that ultimately heightens safety in intricate settings.

In this chapter, we first discuss how we can use knowledge awareness to synthesize reactive controllers. Next, we discuss possible research directions on epistemic logic for multiple agents.

2.1 Knowledge Awareness for Single-agent System

For the automotive scenario mentioned before, a common strategy is adopting an abstract approach to handle the intricate task of processing raw sensor data. This data is crucial for identifying and tracking objects within a given environment. The abstract representation utilized treats perception as symbolic inputs, as discussed in works by Kress et al. (2018)[1], Nilsson (2015)[2], and Schillinger et al. (2018) [3]. Building upon this foundation, our approach involves the application of description logic (DL) to define knowledge and use symbolic computation methods to generate and synthesize reactive controllers tailored for autonomous driving scenarios, as outlined in our previous work [4]. In this context, we introduce a method for seamlessly integrating a *knowledge base* into a reactive controller. This integration includes encoding symbolic awareness of the *entities* and *rules* present in the operational environment. A block-diagram of the proposed methodology is shown in Fig. 2.1.

2.1.1 Description Logic

Description logics (DLs) constitute a group of languages for representing knowledge in a structured and formal manner within an application domain [5]. The term “description logics” is derived from the core idea that concepts within the domain are articulated through concept descriptions. These descriptions are constructed using atomic concepts (unary predicates) and atomic roles (binary predicates) along with concept and role constructors specific to the DL being used. Let’s consider a scenario involving road networks within a road intersection that facilitates the creation of a human-readable, qualitative scene description, and enables autonomous navigation through the

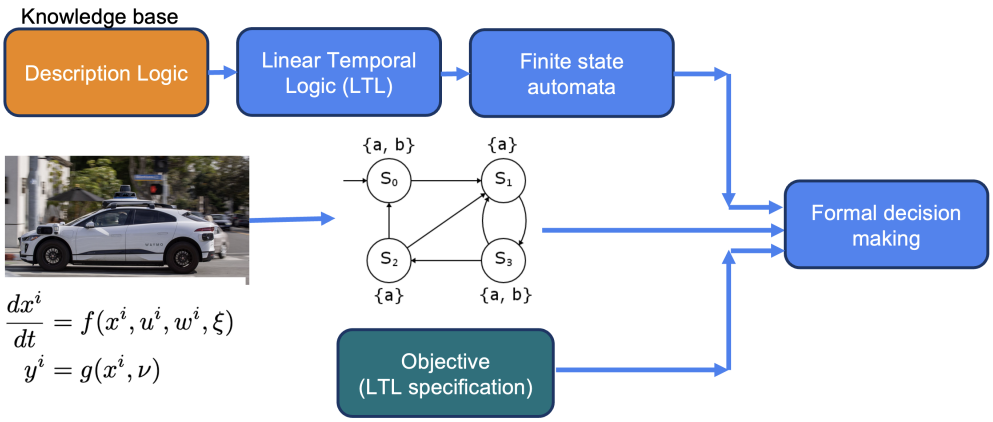

Figure 2.1: The proposed architecture for integrating knowledge awareness into a reactive controller synthesis.

intersection. We will describe a specific *Highway* scenario as "A highway that does not have a divider, contains at least one lane, and is a part of either motorway or transition road". This idea can be articulated using the following concept description,

$$Highway \sqcap \neg Divider \sqcap (\exists hasPart.Lane) \sqcap (\forall isPart.(Motorway \sqcup TransitionRoad)).$$

This description utilizes Boolean constructors such as *conjunction* ($\sqcap$), representing set intersection, *disjunction* ($\sqcup$), representing set union, and *negation* ($\neg$), representing set complement. Additionally, it employs *existential restriction* ($\exists r.C$) and *value restriction* ($\forall r.C$) constructors. Lets say a National Highway is categorized under $\exists hasPart.Lane$, if there exists at least one lane connected to the highway and also falls under the category *Lane*. Similarly, the National Highway falls under $\forall isPart.(Motorway \sqcup TransitionRoad)$ is all of the highway is either motorway or a transition road.

Concept descriptions serve as the foundation for constructing statements within a knowledge base in DL. Such a knowledge base comprises two main components: a terminological part and an assertional part. In the terminological part, referred to as the TBox, relevant concepts and roles within the application domain are described, along with their properties and relationships. TBox statements can include the introduction of names or abbreviations for complex descriptions, such as the concept of simple *Road*, which can be either one way or two way; a *TwoWayRoad* which are not one way; and the concept *National Highway* as an abbreviation for a specific concept description from the previous example

$$\begin{aligned} Road \equiv & OneWayRoad \sqcup TwoWayRoad, \\ TwoWayRoad \sqsubseteq & \neg OneWayRoad, \\ NationalHighway \equiv & Highway \sqcap \neg Divider \sqcap (\exists hasPart.Lane) \\ & \sqcap (\forall isPart.(Motorway \sqcup TransitionRoad)). \end{aligned}$$

The assertional part of the knowledge base, known as the ABox, serves to describe specific

scenarios by specifying attributes of individual entities,

$$NationalHighway(EastWestHighway);\ hasPart(EastWestHighway, Lane66);$$
$$\neg Motorway(DevenportPlatz).$$

The above expressions describe *EastWestHighway* belongs to the concept *NationalHighway*, and *Lane66* is one of the lane in *EastWestHighway*, and *DevenportPlatz* is not a *Motorway*.

A Basic Description Logic

We will outline the syntax and semantics of the fundamental DL $\mathcal{ALC}$. $\mathcal{ALC}$, which stands for "Attributive concept Language with Complements" was initially introduced in [6]. In this section we will provide the formal definitions of the syntax and semantics of the constructors outlined informally in the introduction. The DL encompassing only this assortment of constructors: conjunction, disjunction, negation, existential restriction, and value restriction is termed $\mathcal{ALC}$.

Definition 2.1.1 ($\mathcal{ALC}$ syntax [5, 7])**.** *Let N_C denote a set comprising concept names and N_R denote a set comprising role names. The set of $\mathcal{ALC}$-concept descriptions is the smallest set satisfying the following conditions:*

1. *$\top$, $\bot$, and every concept name $A \in N_C$ are considered $\mathcal{ALC}$-concept descriptions.*
2. *If C and D are $\mathcal{ALC}$-concept descriptions and $r \in N_R$, then $C \sqcap D$, $C \sqcup D$, $\neg C$, $\forall r.C$, and $\exists r.C$ are also $\mathcal{ALC}$-concept descriptions.*

In the subsequent discussion, we will frequently refer "$\mathcal{ALC}$-concept" in place of "$\mathcal{ALC}$-concept description". The interpretation of $\mathcal{ALC}$ (as well as DLs in general) is given in terms of interpretations.

Definition 2.1.2 ($\mathcal{ALC}$ semantics [5, 7])**.** *An interpretation $\mathcal{I} = (\Delta^{\mathcal{I}}, .^{\mathcal{I}})$ comprises a nonempty set $\Delta^{\mathcal{I}}$ referred to as the domain of $\mathcal{I}$, and a function $.^{\mathcal{I}}$ that maps every $\mathcal{ALC}$-concept to a subset of $\Delta^{\mathcal{I}}$ and every role name to a subset of $\Delta^{\mathcal{I}} \times \Delta^{\mathcal{I}}$ such that, for all $\mathcal{ALC}$ concepts C, D and all role names r*

$$\top^{\mathcal{I}} = \Delta\mathcal{I};\ \bot^{\mathcal{I}} = \varnothing;\ (C \sqcap D)^{\mathcal{I}} = C^{\mathcal{I}} \cap D^{\mathcal{I}};\ (C \sqcup D)^{\mathcal{I}} = C^{\mathcal{I}} \cup D^{\mathcal{I}};\ \neg C = \Delta^{\mathcal{I}} \backslash C^{\mathcal{I}};$$
$$(\exists r.C) = \{x \in \Delta^{\mathcal{I}} \mid \textit{There is some } y \in \Delta^{\mathcal{I}} \textit{ with } \langle x, y\rangle \in r^{\mathcal{I}} \textit{ and } y \in C^{\mathcal{I}}\};$$
$$(\forall r.C) = \{x \in \Delta^{\mathcal{I}} \mid \textit{For all } y \in \Delta^{\mathcal{I}} \textit{ if } \langle x, y\rangle \in r^{\mathcal{I}} \textit{ then } y \in C^{\mathcal{I}}\}.$$

We define $C^{\mathcal{I}}(r^{\mathcal{I}})$ as the extension of the concept C (role name r) within the interpretation $\mathcal{I}$. If $x \in C^{\mathcal{I}}$, then x is an instance of C in $\mathcal{I}$.

As previously discussed, a DL knowledge base (KB) comprises two components: a terminological part, known as the TBox, and an assertional part, known as the ABox. Each part contains a collection of axioms. The most general form of TBox axioms is represented by general concept inclusions.

Definition 2.1.3 (TBox [5, 7])**.** *A general concept inclusion (GCI) takes the form $C \sqsubseteq D$, where C and D are $\mathcal{ALC}$-concepts. A finite set of GCIs is referred to as a TBox. An interpretation $\mathcal{I}$ is considered a model of a GCI $C \sqsubseteq D$ if $C^{\mathcal{I}} \subseteq D^{\mathcal{I}}$. $\mathcal{I}$ is a model of a TBox $\mathcal{T}$ if it satisfies every GCI within $\mathcal{T}$.*

We denote the symmetrical pair of GCIs $C \sqsubseteq D$ and $D \sqsubseteq C$ with the abbreviation $C \equiv D$. An axiom in the form $A \equiv C$, where A represents a concept name, is termed a definition. A TBox $\mathcal{T}$ is labeled definitorial if it solely comprises definitions, under the additional constraints that (i) $\mathcal{T}$ includes at most one definition for any given concept name, and (ii) $\mathcal{T}$ is acyclic, meaning the definition of any concept A in $\mathcal{T}$ does not refer (directly or indirectly) to A itself. Such TBoxes are also referred to as acyclic TBoxes in literature. In a definitorial TBox $\mathcal{T}$, concept names appearing on the left-hand side of a definition are termed defined concepts, while the others are referred to as primitive concepts.

The ABox consist of two types of axioms: one for asserting that an individual is an instance of a specified concept, and the other for asserting that a pair of individuals is an instance of a given role name.

Definition 2.1.4 (ABox [5, 7])**.** *An assertional axiom takes the form $x : C$ or $(x, y) : r$, where C represents an $\mathcal{ALC}$-concept, r is a role name, and x and y are individual names. A finite collection of assertional axioms is termed an ABox. In an interpretation $\mathcal{I}$, an assertional axiom $x : C$ is deemed satisfied if $x^{\mathcal{I}} \in C^{\mathcal{I}}$, and an assertional axiom $(x, y) : r$ holds true if $\langle x^{\mathcal{I}}, y^{\mathcal{I}} \rangle \in r^{\mathcal{I}}$. Furthermore, $\mathcal{I}$ is considered a model of an ABox $\mathcal{A}$ if it satisfies every axiom within $\mathcal{A}$.*

Now, the Knowledge base can be fined as:

Definition 2.1.5 (Knowledge base (KB) [5, 7])**.** *A knowledge base (KB) is represented as a pair $(\mathcal{T}, \mathcal{A})$, where $\mathcal{T}$ denotes a TBox and $\mathcal{A}$ represents an ABox. An interpretation $\mathcal{I}$ is considered a model of a KB $\mathcal{K} = (\mathcal{T}, \mathcal{A})$ if $\mathcal{I}$ satisfies both $\mathcal{T}$ and $\mathcal{A}$.*

We can write $\mathcal{I} \models \mathcal{K}$, respectively $\mathcal{I} \models \mathcal{T}$, $\mathcal{I} \models \mathcal{A}$, $\mathcal{I} \models a$ to denote that $\mathcal{I}$ is a model of a KB $\mathcal{K}$ respectively TBox $\mathcal{T}$, ABox $\mathcal{A}$, axiom a.

Important Inference Problems

In this section the inference problems with respect to a KB consisting of a TBox and an ABox is defined.

Definition 2.1.6 (Inference Problem [5, 7])**.** *Given a KB $\mathcal{K} = (\mathcal{T}, \mathcal{A})$, where $\mathcal{T}$ is a TBox and $\mathcal{A}$ is an ABox, $\mathcal{K}$ is called consistent if it has a model. A concept C is called satisfiable with respect to $\mathcal{K}$ if there is a model $\mathcal{I}$ of $\mathcal{K}$ with $C^{\mathcal{I}} = \varnothing$. Such an interpretation is called a model of C with respect to $\mathcal{K}$. The concept D subsumes the concept C with respect to $\mathcal{K}$ (written $\mathcal{K} \models C \sqsubseteq D$) if $C^{\mathcal{I}} \subseteq D^{\mathcal{I}}$ holds for all models $\mathcal{I}$ of $\mathcal{K}$. Two concepts C,D are equivalent with respect to $\mathcal{K}$ (written $\mathcal{K} \models C \equiv D$) if they subsume each other with respect to $\mathcal{K}$. An individual a is an instance of a concept C with respect to $\mathcal{K}$ (written as $\mathcal{K} \models a : C$) if $a^{\mathcal{I}} \in C^{\mathcal{I}}$ holds for all models $\mathcal{I}$ of $\mathcal{K}$. A pair of individuals (a, b) is an instance of a role name r with respect to $\mathcal{K}$ (written $\mathcal{K} \models (a, b) : r$) if $\langle a^{\mathcal{I}}, b^{\mathcal{I}} \rangle \in r^{\mathcal{I}}$ holds for all models $\mathcal{I}$ of $\mathcal{K}$.*

Relation between DL and First-order Logic

Many DLs can be viewed as subsets of first-order predicate logic. The primary rationale for employing Description Logics instead of general first-order predicate logic in knowledge representation is that most DLs constitute decidable subsets of first-order predicate logic. In

other words, there exist effective methods for resolving the inference problems that arise within these DLs.

Viewing role names as binary relations and concept names as unary relations, we define two translation functions, π_x and π_y, that inductively map $\mathcal{ALC}$-concepts into first order formulae with one free variable, x or y [5, 7]:

$$\begin{aligned}
\pi_x(A) &= A(x), & \pi_x(y) &= A(y),\\
\pi_x(C \sqcap D) &= \pi_x(C) \wedge \pi_x(D), & \pi_y(C \sqcap D) &= \pi_y(C) \wedge \pi_y(D),\\
\pi_x(C \sqcup D) &= \pi_x(C) \vee \pi_x(D), & \pi_y(C \sqcup D) &= \pi_y(C) \vee \pi_y(D),\\
\pi_x(\exists r.C) &= \exists y.r(x,y) \wedge \pi_y(C), & \pi_y(\exists r.C) &= \exists x.r(y,x) \wedge \pi_x(C),\\
\pi_x(\forall r.C) &= \forall y.r(x,y) \Rightarrow \pi_y(C), & \pi_y(\forall r.C) &= \forall x.r(y,x) \Rightarrow \pi_x(C).
\end{aligned}$$

Given this, we can translate a TBox $\mathcal{T}$ and an ABox A as follows, where $\psi[x/a]$ denotes the formula obtained from ψ by replacing all free occurrences of x with a:

$$\begin{aligned}
\pi(\mathcal{T}) &= \bigwedge_{C \sqsubseteq D \in \mathcal{T}} \forall x.(\pi_x(C) \Rightarrow \pi_x(D)),\\
\pi(\mathcal{A}) &= \bigwedge_{a:C \in \mathcal{A}} \pi_x(C)[x/a] \wedge \bigwedge_{(a,b):r \in \mathcal{A}} r(a,b).
\end{aligned}$$

This translation preserves the semantics. Consequently, we find that reasoning in DLs aligns with first-order inference.

Theorem 1 (Baader et al. [5, 7])**.** *Let $(\mathcal{T}, \mathcal{A})$ be an $\mathcal{ALC}$-knowledge base, C, D possibly complex $\mathcal{ALC}$-concepts, and a an individual name. Then*

1. *$(\mathcal{T}, \mathcal{A})$ is consistent iff $\pi(\mathcal{T}) \wedge \pi(\mathcal{A})$ is consistent,*
2. *$(\mathcal{T}, \mathcal{A}) \models C \sqsubseteq D$ iff $(\pi(\mathcal{T}) \wedge (\pi(\mathcal{A})) \Rightarrow (\pi(\{C \sqsubseteq D\}))$ is valid,*
3. *$(\mathcal{T}, \mathcal{A}) \models a : C$ iff $(\pi(\mathcal{T}) \wedge (\pi(\mathcal{A})) \Rightarrow (\pi(\{a : C\}))$ is valid.*

This translation not only provides an alternative way of defining the semantics of $\mathcal{ALC}$, but also tells us that all the introduced reasoning problems for $\mathcal{ALC}$ knowledge bases are decidable. In fact, the translation of a knowledge base uses only variables x and y, and thus yields a formula in the two variable fragment of first-order logic, which is known to be decidable in non-deterministic exponential time [8].

2.1.2 Knowledge Awareness

We consider the symbolic representation of *knowledge* about a *fact* or property, denoted as $M \models \phi$. Here, ϕ represents the property symbolically as a Boolean formula, capable of being evaluated as either true or false, and M represents a set of truth assignments to the atomic propositions in ϕ. Essentially, knowledge can be seen as a declaration of the truth of ϕ in M. Symbolic *awareness* of ϕ is seen as understanding the formula, both syntactically and semantically, indicating the ability to evaluate it, although not necessarily immediately. An *awareness knowledge base* Ω comprises various distinct symbolic awareness formulas. We assume Ω is expressed as DL statements and utilize its

computationally efficient implementation for the classification task, i.e., determining if a concrete symbolic input M matches a known symbolic description ϕ. Consequently, this corresponds to the decision process for $M \models \phi$. If $M \models \phi$ evaluates to true, we infer that the detected entity M is an instance of object ϕ [4].

2.1.3 Linear Temporal Logic Specification

We convert the set of roles in Description Logic (DL) into a collection of Linear Temporal Logic (LTL) specifications to integrate it into the symbolic computation of the controller. The transformation of a DL model Ω into a series of LTL formulas is straightforward, representing subclassing relations between a shared property symbol and a sharing property symbol. These relationships are then considered equivalent and can be directly translated into a set of logical implications between the involved symbolic variables: if symbol a is a subclass of symbol b, the relation is logically encoded as $a \Rightarrow b$. We denote σ as the set of LTL formulas derived from the translation of the semantic relations in Ω. It is worth noting that this process can also be automated by a compiler to streamline this step.

2.1.4 Finite Transition System

A finite transition system (FTS) is defined as a tuple $T = (Q, \theta, \Sigma, \Pi, \tau, O)$, where:

- Q is a finite set of states.
- $\theta \subset Q$ represents the set of possible initial states.
- $\Sigma \subseteq Q \times Q$ denotes the set of state transitions.
- $\Pi := p_1, \ldots, p_n$ is a collection of state properties represented as atomic propositions.
- $\tau : Q \to 2^{\Pi}$ is a labeling function that associates properties with each state.
- O is a function that maps each 'edge,' denoted as $e ::= q \to q\prime$ where $(q, q\prime) \in \Sigma$, to a corresponding output $y \in Y$.

A FTS describes a collection of state sequences, encompassing all the possible paths through it. With labels marking the states along these paths, we constrain the permissible (motion) paths by articulating temporal logic formulas concerning those labels.

2.1.5 Generalized Reactivity

For the automated creation of a reactive controller, a polynomial-time algorithm developed by Piterman et al. [9] is employed. This algorithm is rooted in principles derived from game theory. It generates the controller as the winning strategy in an infinite-duration, turn-based game of a specific type known as *generalized reactivity* (GR(1)). In this game model, the system is pitted against an uncontrolled environment acting as its adversary. The goal is to find a solution where, irrespective of the environment's subsequent move, the system always possesses a move that guarantees the fulfillment of its safety specifications.

Problem statement: Given the model of the system as a finite transition system, an LTL specification Φ, and a knowledge base Ω encoding awareness about the unknown environment,

design automatically a reactive controller to satisfy the specification using the knowledge base and the perception from the environment.

2.1.6 Symbolic Controller Synthesis

The context-aware controller is developed through the Tulip tool [10] for generating reactive controllers based on GR(1) specifications. This process necessitates an FTS to represent the system's spatial motion capabilities and a set of LTL specifications as shown in Fig. 2.1. Within this system, the concept of 'awareness' is integrated of the operating environment, enabling it to identify and comprehend familiar objects [4]. Additionally, it equips the system to handle the partial symbolic characteristics of these objects, facilitating the gradual adaptation of its responses.

2.2 Epistemic Logic for Multi-agent Systems

Epistemic logic for multi-agent systems is a branch of formal logic that deals with reasoning about knowledge, belief, and other mental states of multiple agents in a distributed environment. It provides a framework for representing and analyzing the information available to different agents, their beliefs about the world, and how these beliefs are updated over time as they interact with each other.

2.2.1 Literature

To improve knowledge awareness of multi-agent systems, we are interested in reasoning about their knowledge though epistemic temporal logics. These languages have been developed to represent a group's knowledge instead of the knowledge of an individual system [11]. When verifying multi-agent systems, epistemic logics are especially used in combination with *interpreted systems* [11, 12, 13, 14, 15, 16, 17] or with *(Discrete Time) Markov Chains* (DTMC) [18]. The former type of systems include agent local states, system global states, and an epistemic accessibility relation indicating knowledge transfer between global states. The later is a similar model as a Markov Decision Process (MDP), however, it contains a probability matrix instead of probabilistic transition function. It should be noted though that most approaches translate the interpreted system description to a DTMC as a form of model reduction before continuing the model checking procedure.

The list of existing epistemic logics include (but is not limited to) Dynamic Epistemic Logic (DAL) [19], Computation Tree Logic of Knowledge (CTLK) [12], and linear time temporal-epistemic logic (LTLK) [17]. Also probabilistic extensions exist, such as K-PCTL [18], PETL [20], and PCTLK [11].

2.2.2 Research Directions

We are working on the following research directions:

- Define quantitative semantics for a class of epistemic logic.
- In view of the problem statement above, extend the available computational approaches from finite-state systems to continuous-state systems.

- Extend from verification to control synthesis.
- Include uncertainties in the form of probabilities in the system dynamics.

We discuss each of these research directions in more detail next.

Quantitative Semantics

Quantitative semantics allow for a quantification of how close (or far) the system is from satisfying (or violating) the temporal logic specification. These techniques have gained a lot of attention recently in the context of the language *signal temporal logic* [21, 22]. This language is used in many applications, such as model predictive control [23, 24, 25] and planning for multi-agent systems [26, 27, 28, 29].

We want to combine the knowledge operators of epistemic logics such as PCTLK [11] with quantitative semantics similar to STL [21]. This allows us to not only verify whether an epistemic specification is satisfied or violated, but also quantify how close it is to satisfying (or violating) the specification. An interesting quantitative semantic is for example space robustness, denoted by $\rho^{\varphi}(x, t)$ that defines how well a signal x satisfies a formula φ at time t.

More specifically, we are interested in describing the following epistemic properties over specifications ϕ:

- $K_i\phi$ representing *agent i knows*,
- $E_G\phi$ representing *every agent in the group G knows*,
- $C_G\phi$ representing *common knowledge in the group G*,
- $D_G\phi$ representing *distributed knowledge in the group G*.

Continuous-state Systems

To accurately describe the behavior of a dynamical agent, we often require a continuous state space. This can be visualized from the car example in Fig. 2.1, which has a continuous dynamics. Though for the analysis in our previous work [4], we have translated dynamical equations of the system into a finite state automata. As indicated in the literature overview, epistemic logics have mainly been used for interpreted systems, which have a finite-state space. To the best of our knowledge, there is no work that performs verification or control synthesis of epistemic temporal logic specifications for systems with a *continuous-state* space. Hence, to achieve knowledge awareness for MAS through epistemic logic for dynamical systems, it is crucial to extend the verification of epistemic temporal logic specifications from finite-state systems to continuous-state systems. Utilizing the symbolic concepts and techniques developed so far in the SymAware project, the most immediate approach is to perform a finite-state abstraction of the continuous-state system and determine how the verification results apply to the original continuous-state system [30].

By extending the research direction on quantitative semantics towards continuous-state systems we will gain very beneficial insight in how robust a dynamical system is with respect to epistemic temporal logic requirements.

Control Synthesis

Instead of focusing on verifying the behavior of a (controlled) system, it is also of interest to consider correct-by-design control synthesis. By doing so, the controlled system satisfies the specification *by design* [31, 32]. By having quantitative semantics in the epistemic temporal logic specification, we can include the robustness parameter into the control design and choose the desired robustness of the controller. Here, with robustness we mean in the sense of how close the controlled system is to violating the specification.

Probabilistic Systems

Finally, we envision the extension towards probabilistic systems. Since, almost all dynamical systems are subject to some probabilistic (or stochastic) influence. This is especially important in the context of safety-critical systems, where a minor mistake can have disastrous consequence and guaranteed safety is crucial. Lets consider a scenario of stochastic pedestrian behavior at a crosswalk associated with Fig 2.1. In this scenario, pedestrians might have varying speeds, unpredictable movements, and different intentions, making it challenging for the autonomous vehicle to accurately predict their actions. Factors such as distractions, sudden changes in direction, ambiguous gestures can introduce uncertainty into the pedestrian's behavior. To address this uncertainty, autonomous vehicles utilize probabilistic models and stochastic approaches. These models incorporate data from sensors such as cameras, lidar, and radar to estimate the probability distribution of possible pedestrian trajectories. The literature on automatic control synthesis of stochastic systems is rich and keeps on developing [33], however, there are no approaches that consider epistemic logic. Even though, this is very important, especially for MAS.

To make the extension towards probabilistic systems, we are inspired by the method detailed in [34], where the authors consider continuous-state probabilistic systems. The essence of the approach is to abstract the support of the probabilistic evolution and formulate a fairness assumption. This approach simplifies the control synthesis procedure significantly. Finally, the control synthesis procedure is solved as a $2\frac{1}{2}$-player game on a finite graph. We plan to incorporate a similar technique to make the extension towards probabilistic systems.

Bibliography

[1] H. Kress-Gazit, M. Lahijanian, and V. Raman, "Synthesis for robots: Guarantees and feedback for robot behavior," *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 1, pp. 211–236, 2018.

[2] P. Nilsson, O. Hussien, A. Balkan, Y. Chen, A. D. Ames, J. W. Grizzle, N. Ozay, H. Peng, and P. Tabuada, "Correct-by-construction adaptive cruise control: Two approaches," *IEEE Transactions on Control Systems Technology*, vol. 24, no. 4, pp. 1294–1307, 2015.

[3] P. Schillinger, M. Bürger, and D. V. Dimarogonas, "Simultaneous task allocation and planning for temporal logic goals in heterogeneous multi-robot systems," *The international journal of robotics research*, vol. 37, no. 7, pp. 818–838, 2018.

[4] A. Calvagna, A. Ghosh, and S. Soudjnai, "Using knowledge awareness to improve safety of autonomous driving," in *2023 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*. IEEE, 2023, pp. 2997–3002.

[5] F. Baader, *The description logic handbook: Theory, implementation and applications*. Cambridge university press, 2003.

[6] M. Schmidt-Schauß and G. Smolka, "Attributive concept descriptions with complements," *Artificial intelligence*, vol. 48, no. 1, pp. 1–26, 1991.

[7] F. Baader, I. Horrocks, and U. Sattler, "Description logics," *Foundations of Artificial Intelligence*, vol. 3, pp. 135–179, 2008.

[8] E. Graedel, P. Kolaitis, and M. Vardi, "On the decision problem for two-variable first-order logic," *Bulletin of Symbolic Logic*, vol. 3, no. 1, pp. 53–69, 1997.

[9] N. Piterman, A. Pnueli, and Y. Saâ€™ar, "Synthesis of reactive (1) designs," in *Verification, Model Checking, and Abstract Interpretation: 7th International Conference, VMCAI 2006, Charleston, SC, USA, January 8-10, 2006. Proceedings 7*. Springer, 2006, pp. 364–380.

[10] T. Wongpiromsarn, U. Topcu, N. Ozay, H. Xu, and R. M. Murray, "TuLiP: a software toolbox for receding horizon temporal logic planning," in *Proceedings of the 14th international conference on Hybrid systems: computation and control*, 2011, pp. 313–314.

[11] W. Wan, J. Bentahar, and A. B. Hamza, "Model checking epistemic–probabilistic logic using probabilistic interpreted systems," *Knowledge-Based Systems*, vol. 50, pp. 279–295, 2013.

[12] W. Penczek and A. Lomuscio, "Verifying epistemic properties of multi-agent systems via bounded model checking," in *Proceedings of the second international joint conference on Autonomous agents and multiagent systems*, 2003, pp. 209–216.

[13] K. Sultan, J. Bentahar, H. Yahyaoui, and R. Mizouni, "Model checking agent-based communities against uncertain group commitments and knowledge," *Expert Systems with Applications*, vol. 177, p. 114792, 2021.

[14] A. Lomuscio, C. Pecheur, and F. Raimondi, "Automatic verification of knowledge and time with nusmv," in *Proceedings of the Twentieth International Joint Conference on Artificial Intelligence*. IJCAI/AAAI Press, 2007, pp. 1384–1389.

[15] A. Lomuscio and W. Penczek, "Symbolic model checking for temporal-epistemic logics," *ACM SIGACT News*, vol. 38, no. 3, pp. 77–99, 2007.

[16] R. van der Meyden and N. V. Shilov, "Model checking knowledge and time in systems with perfect recall," in *International Conference on Foundations of Software Technology and Theoretical Computer Science*. Springer, 1999, pp. 432–445.

[17] A. Meski, W. Penczek, and M. Szreter, "Bounded model checking for linear time temporal-epistemic logic," in *2012 Imperial College Computing Student Workshop*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik, 2012.

[18] C. Delgado and M. Benevides, "Verification of epistemic properties in probabilistic multi-agent systems," in *German Conference on Multiagent System Technologies*. Springer, 2009, pp. 16–28.

[19] S. L. Uckelman, "Deceit and indefeasible knowledge: The case of dubitatio," *Journal of Applied Non-Classical Logics*, vol. 21, no. 3-4, pp. 503–519, 2011.

[20] Z. Cao, "Model checking for epistemic and temporal properties of uncertain agents," in *Pacific Rim International Workshop on Multi-Agents*. Springer, 2006, pp. 46–58.

[21] O. Maler and D. Nickovic, "Monitoring temporal properties of continuous signals," in *International Symposium on Formal Techniques in Real-Time and Fault-Tolerant Systems*. Springer, 2004, pp. 152–166.

[22] A. Donzé, "On signal temporal logic," in *Runtime Verification: 4th International Conference, RV 2013, Rennes, France, September 24-27, 2013. Proceedings 4*. Springer, 2013, pp. 382–383.

[23] V. Raman, A. Donzé, M. Maasoumy, R. M. Murray, A. Sangiovanni-Vincentelli, and S. A. Seshia, "Model predictive control with signal temporal logic specifications," in *53rd IEEE Conference on Decision and Control*. IEEE, 2014, pp. 81–87.

[24] S. Sadraddini and C. Belta, "Robust temporal logic model predictive control," in *2015 53rd Annual Allerton Conference on Communication, Control, and Computing (Allerton)*. IEEE, 2015, pp. 772–779.

[25] T. Yang, Y. Zou, S. Li, X. Yin, and T. Jia, "Signal temporal logic synthesis under model predictive control: A low complexity approach," *Control Engineering Practice*, vol. 143, p. 105782, 2024.

[26] D. Sun, J. Chen, S. Mitra, and C. Fan, "Multi-agent motion planning from signal temporal logic specifications," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 3451–3458, 2022.

[27] A. T. Buyukkocak, D. Aksaray, and Y. Yazıcıoğlu, "Planning of heterogeneous multi-agent systems under signal temporal logic specifications with integral predicates," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 1375–1382, 2021.

[28] L. Lindemann, "Planning and control of multi-agent systems under signal temporal logic specifications," Ph.D. dissertation, KTH Royal Institute of Technology, 2020.

[29] L. Lindemann and D. V. Dimarogonas, "Decentralized control barrier functions for coupled multi-agent systems under signal temporal logic tasks," in *2019 18th European Control Conference (ECC)*. IEEE, 2019, pp. 89–94.

[30] E. M. Clarke, O. Grumberg, and D. E. Long, "Model checking and abstraction," *ACM transactions on Programming Languages and Systems (TOPLAS)*, vol. 16, no. 5, pp. 1512–1542, 1994.

[31] C. Belta, B. Yordanov, and E. A. Gol, *Formal methods for discrete-time dynamical systems*. Springer, 2017, vol. 15.

[32] P. Tabuada, *Verification and control of hybrid systems: a symbolic approach*. Springer Science & Business Media, 2009.

[33] A. Lavaei, S. Soudjani, A. Abate, and M. Zamani, "Automated verification and synthesis of stochastic hybrid systems: A survey," *Automatica*, vol. 146, p. 110617, 2022.

[34] R. Majumdar, K. Mallik, A.-K. Schmuck, and S. Soudjani, "Symbolic control for stochastic systems via finite parity games," *Nonlinear Analysis: Hybrid Systems*, vol. 51, p. 101430, 2024.